



S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR.

PreLab

Aim: To Hello TensorFlow.js Verify TensorFlow.js installation by creating tensors, performing addition, and printing results.

Name of Student:

Roll No .:

Semester/Year: VI / III

Academic Session: 2025-26

Date of Performance:

Date of Submission:

AIM: Hello TensorFlow.js Verify TensorFlow.js installation by creating tensors, performing addition, and printing results.

OBJECTIVE/EXPECTED LEARNING OUTCOME:

- To understand what TensorFlow.js is and its basic purpose.
- To learn how to create tensors using TensorFlow.js.
- To perform basic tensor operations such as addition.
- To verify that TensorFlow.js is correctly installed and working.
- To display and interpret tensor results.

THEORY:

TensorFlow.js is an open-source JavaScript library developed by Google that allows developers to build, train, and run machine learning models directly in the browser or using Node.js.

In TensorFlow.js, data is stored in a special structure called a Tensor.
A tensor is a multi-dimensional array used to represent numerical data.

Basic operations like addition, subtraction, multiplication, etc., can be performed on tensors. If these operations execute correctly, it confirms that TensorFlow.js is properly installed and functioning.

In this pre-lab experiment:

- We create two tensors
- Add them using TensorFlow.js
- Print the result to verify installation

Program:

```
!npm install @tensorflow/tfjs

%%writefile hello_tfjs.js

// Import TensorFlow.js
const tf = require('@tensorflow/tfjs');

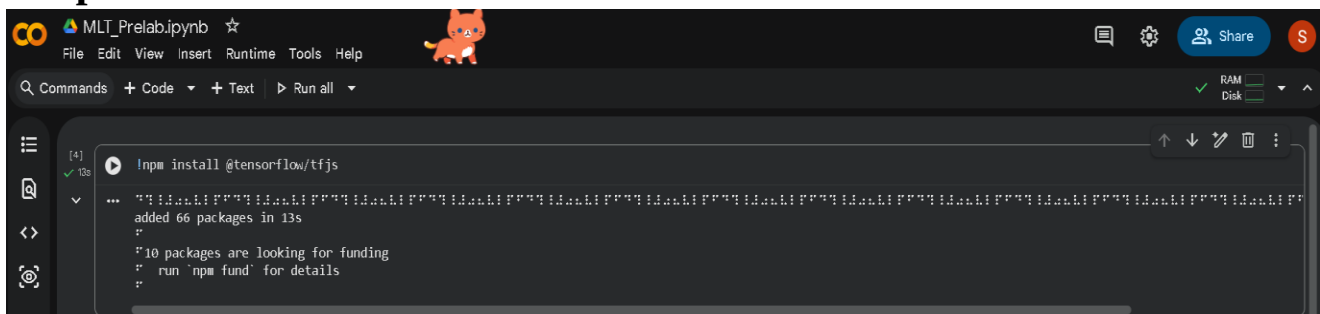
// Create tensors
const tensor1 = tf.tensor([1, 2, 3]);
const tensor2 = tf.tensor([4, 5, 6]);

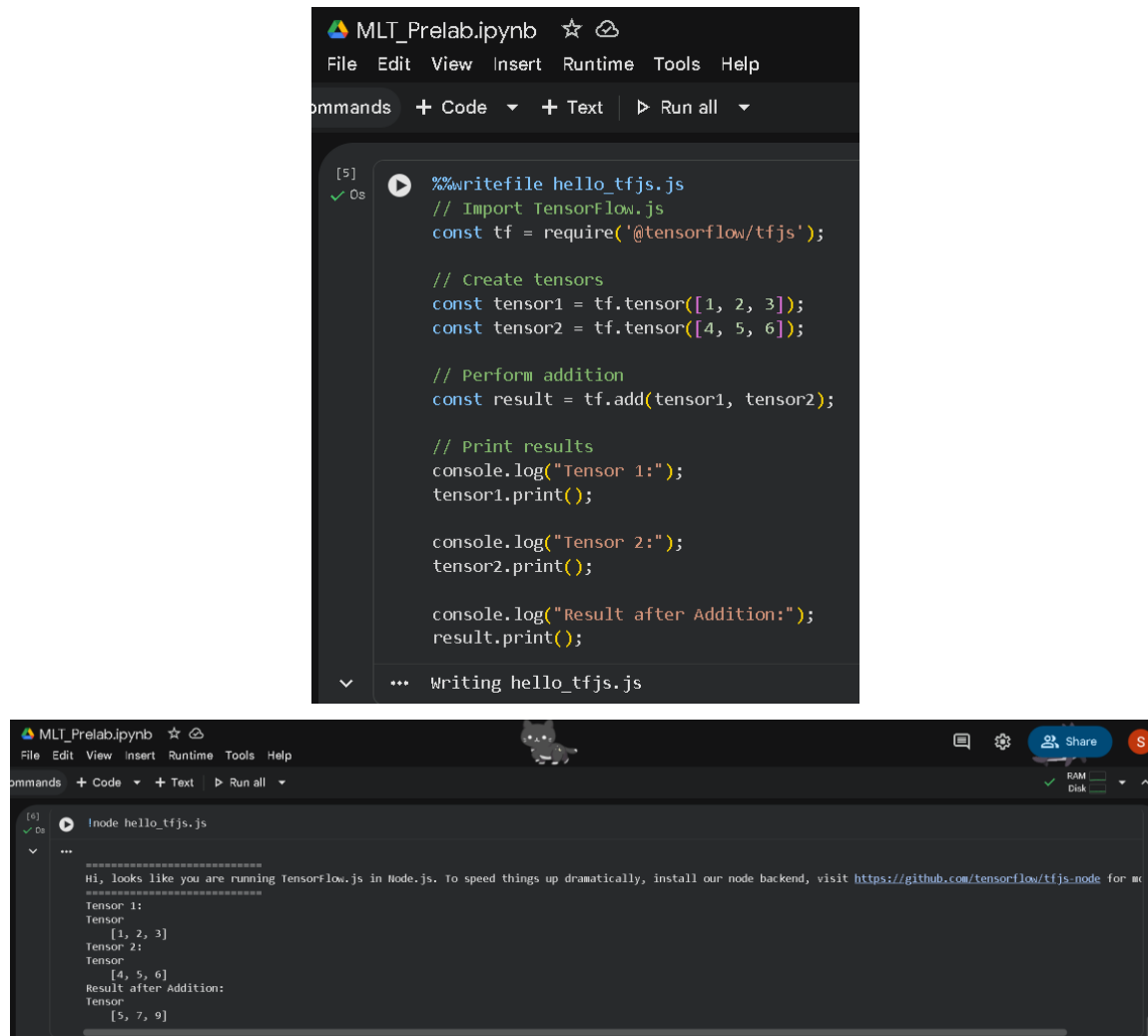
// Perform addition
const result = tf.add(tensor1, tensor2);

// Print results
console.log("Tensor 1:");
tensor1.print();
console.log("Tensor 2:");
tensor2.print();
console.log("Result after Addition:");
result.print();

!node hello_tfjs.js
```

Output:

A screenshot of a Jupyter Notebook interface. The top bar shows the notebook name 'MLT_Prelab.ipynb' and various icons for file operations, settings, and sharing. The main area displays a code cell with the command '!npm install @tensorflow/tfjs'. Below the code, the output is shown, indicating that 66 packages were added in 13 seconds and that 10 packages are looking for funding. The interface includes a sidebar with icons for file explorer, search, and other notebook functions.



The image displays two screenshots of a Jupyter Notebook interface, specifically the 'MLT_Prelab.ipynb' file. The top screenshot shows the code editor with the following JavaScript code:

```
%%writefile hello_tfjs.js
// Import TensorFlow.js
const tf = require('@tensorflow/tfjs');

// Create tensors
const tensor1 = tf.tensor([1, 2, 3]);
const tensor2 = tf.tensor([4, 5, 6]);

// Perform addition
const result = tf.add(tensor1, tensor2);

// Print results
console.log("Tensor 1:");
tensor1.print();

console.log("Tensor 2:");
tensor2.print();

console.log("Result after Addition:");
result.print();
```

The bottom screenshot shows the output of the code execution. It indicates that the file 'hello_tfjs.js' was written and then executed. The output text is as follows:

```
Hi, looks like you are running TensorFlow.js in Node.js. To speed things up dramatically, install our node backend, visit https://github.com/tensorflow/tfjs-node for more info.

Tensor 1:
Tensor
[1, 2, 3]
Tensor 2:
Tensor
[4, 5, 6]
Result after Addition:
Tensor
[5, 7, 9]
```

Conclusion

In this pre-lab experiment, TensorFlow.js was successfully verified by creating tensors, performing tensor addition, and printing the output. The correct result confirms that TensorFlow.js is properly installed and ready for performing machine learning operations.